
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 1: Metamodelling
Project Type: Code Generation
September 25, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Introduction and Scope	1
2	Domain Analysis	1
2.1	Observations From the Existing Configuration	1
2.2	Structural and Operational Concepts	1
2.3	Notation	1
3	Modelling Decisions	1
3.1	Three Model Stages, Two Metamodels	2
3.2	What Each Stage May Contain	2
3.3	Shared Values Are Set by the Platform	2
3.4	Derived Values Cannot Be Overridden	2
3.5	Requirements Instead of Target Fields	2
3.6	Enumerations Instead of Free-Form Strings	2
3.7	Release Grouping and Deployment Order	2
4	Alternatives Considered	2
4.1	A Third Metamodel for the Generated Files	2
4.2	Grammar First or Metamodel First	2
4.3	One Resolved Model, or One Per Application	3
5	The Source Metamodel	3
5.1	The Project Document	3
5.2	The Platform Document	6
5.3	Enumerations	8
6	The Target Metamodel	8
6.1	Required Content	8
6.2	Structure	8
6.3	The Path Plan	10
6.4	The Generated Resources	10
6.5	Keeping Target Field Names Out of the Model	10
7	Implementation in EMF	10
7.1	The Concrete Syntax	10
7.2	The Constraints	10
7.3	How an Invalid Model Is Reported	11
8	Example Models	11
8.1	The Minimal Model	12
8.2	A Resolved Model as a Deployment View	12
8.3	Example Target Models	12

A	Worked Examples	13
A.1	The Minimal Model, End to End	13
A.2	Continuous and Interrupted Applications in One Project	13
A.3	A Rejected Model	13
A.4	The Hand-in Archive	13
B	All Constraints	15

1 Introduction and Scope

Task 0 proposed generating Kubernetes deployment files from application requirements. Task 1 defines two Ecore metamodels: the authored source language and the *resolved model*, which records platform-derived values. OCL constraints, Xtext syntax and example models accompany them. Appendix A.4 describes the hand-in archive.

The Task 0 feedback led to a smaller scope (Table 1) and to the notation of Section 2.3.

In scope	Out of scope
Two metamodels: the authored source language, and the resolved model.	A metamodel of the generated files. They are text, and are compared as text.
A closed set of six generators for one cluster. Adding a generator requires a recorded decision.	Additional target platforms, and a second cluster.
Validation of a source model, and the constraints that reject an invalid one.	Applying the generated files to a cluster.
Dependency resolution between applications.	Runtime behaviour, scaling decisions and monitoring rules.

Table 1: Scope of the Project After the Task 0 Feedback

Correctness here concerns the generated model and files, which are not applied to a cluster.

2 Domain Analysis

The existing configuration covers one Kubernetes cluster, seven nodes, ten namespaces, about thirty processes and twenty applications. It is maintained by hand.

2.1 Observations From the Existing Configuration

Repeated values. One hostname appears seven times across three repositories. Two tests check those copies for disagreement. A port appears in the container specification, service, probe and monitoring rule.

Repeated derivations. Four deployments contain the same rollout block. Comments explain that recreate leaves a period without a pod and a JVM needs more startup time than a static bundle. A *derivation* computes such values from the authored model and cluster policy.

Mechanisms instead of requirements. The files specify rollout strategies and node labels. They do not state whether an upgrade may interrupt service or what a process needs from a machine. A volume that cannot attach twice requires recreate. The error appears only when a second node is used.

Shared values. Hostnames must be unique. One process uses a graphics card at a time. The *edge* consists of shared entry points for external traffic (Section 3.3).

2.2 Structural and Operational Concepts

Structurally, a project groups applications. An application groups processes released together. Processes offer and use named ports. Hostname paths lead to an application's processes.

Operational concepts specify machine requirements, data durability, secret access and delivery, health checks, startup time and whether versions may run together during an upgrade.

2.3 Notation

The Task 0 feedback suggested ArchiMate or the C4 model instead of, or beside, UML class diagrams.

A metamodel defines abstract syntax. Ecore implements EMOF classes, attributes, references and multiplicities. Class diagrams show them directly. ArchiMate and C4 describe systems.

For a model instance, Figure 4 shows a deployment view. The resolved model records machines, hostnames and dependencies. Task 3 could generate an ArchiMate technology view or C4 deployment diagram from it.

3 Modelling Decisions

Seven decisions define the metamodels.

3.1 Three Model Stages, Two Metamodels

The pipeline transforms an authored model into a resolved model, then generates text. The first two have metamodels (Section 4.1).

These stages are not metalevels. No `instanceOf` relation connects them.

3.2 What Each Stage May Contain

A *decision* is a platform-computed value, such as eligible machines. **Authors state requirements, not target mechanisms (Section 2.1). Generators use the resolved model without computing decisions.** The resolved model records computed values.

An earlier two-stage design produced three incompatible documents with one version identifier. The resolved model records computed values separately from the authored document.

3.3 Shared Values Are Set by the Platform

The platform sets cluster-unique values and allocations of shared resources. Applications declare other values, which the transformation copies unchanged.

Applications state memory and CPU requirements so processes reserve capacity. The platform compares each requirement with individual machines. It does not sum reservations across the cluster, so it cannot detect aggregate overcommit.

3.4 Derived Values Cannot Be Overridden

The platform derives values from the authored model and platform document. There is no override mechanism (Table 2).

One process had a shorter hand-set deadline because it started faster. Its startup budget already provides the input for a deadline rule.

3.5 Requirements Instead of Target Fields

Attributes describe requirements, such as writable paths, instead of target fields such as a security context (Section 2.1). Generators contain target field names. A check rejects target field names in metamodel keys.

3.6 Enumerations Instead of Free-Form Strings

Fixed sets of values use enumerations. Constraints can cover their literals. Free-form strings have no finite set to check (Table 2).

3.7 Release Grouping and Deployment Order

An application groups processes released together. The dependency graph determines application order. Using that graph for release grouping joined all applications into one unit through transitive dependencies.

4 Alternatives Considered

Table 2 lists the rejected alternatives.

4.1 A Third Metamodel for the Generated Files

A third metamodel could support structural output checks, but a maintained copy of the target schema could drift. It adds no constraint beyond the resolved metamodel. Tests compare generated text with reviewed output and validate it against target schemas. The resolved metamodel contains typed resource classes (Section 6.4).

4.2 Grammar First or Metamodel First

The production implementation parses another serialisation of the source language. If the Xtext grammar defined the language, the two syntaxes could diverge. Both implementations instead check against the metamodel's exported, machine-readable structure.

Alternative	Why it was rejected
Template the target files directly (Helm, Kustomize overlays)	No abstract model supports constraints or shared derivations. Repeated values remain.
Annotate the target files with extra fields	The existing configuration gave authored and resolved forms the same name. Errors could not identify which form was wrong.
Two stages, with the generator resolving values internally	Generator decisions appear in no reviewable model.
A third metamodel for the generated files	It would restate the target schema.
A grammar first, with the metamodel inferred	Two syntaxes need the same abstract concepts.
Free-form strings instead of enumerations	The existing free-form field was wrong in every application.
An override mechanism for derived values	Per-process exceptions would conceal an incorrect general rule.

Table 2: Alternatives Considered and Why Each Was Rejected

4.3 One Resolved Model, or One Per Application

The resolved model covers all applications because edge routing, application order and reverse dependencies require cluster-wide inputs. Separate per-application computations could disagree or omit values.

Each repository receives a *projection*: its application’s part of the resolved model, selected by filtering.

5 The Source Metamodel

The source metamodel is `model/project-intent.ecore` in the archive. A *project* document specifies application requirements. One *platform* document specifies cluster facts and policies (Section 3.3).

5.1 The Project Document

An application using a database declares a migration. One application migrates each project’s database. `Migration` has three subclasses. The platform applies a `ManagedMigration` changelog before the new version starts. `SelfMigration` leaves schema changes to the image. `NoMigration` leaves them to another application.

`cutover` specifies whether service continues during an upgrade. For `continuous`, the new version receives traffic after the application’s processes pass metric checks. This is a *blue/green* switch. For `interrupted`, the old version stops first. `lifecycle` is `application`, `job` or `prepare`. A *prepare process* performs one-off setup after migration and before the new version starts. It serves no traffic.

An `Exposure` belongs to an application because its hostname can route different paths to different processes. Routes refer to their application’s `Surface`. Dependencies refer to another application’s `Surface`.

Figure 1 shows the classes in Table 3 and their relationships.

Class	What it represents
Project	The authored document: an owner and the applications it groups.
Application	A unit released together, with a cluster-unique identifier.
Process	One containerised process: image, kind, ports, needs and health.
Surface	A named port a process offers, declared once.
Exposure	A hostname the application is reachable at, and who may reach it.
Route	A path on that hostname, and the process and surface that serve it.
Placement	What a process needs from a machine, as hard requirements.
Volume	Process storage and its durability class, which sets how loss is handled.
Probe	The endpoint that serves a process's readiness or liveness check.
DependencyEdge	A link to another application's named surface. Where the provider has databases, it states how the derived database credential may rotate.
Credentials	The rotation an application supports for the database credential derived for its dependency.
Migration	How the schema is migrated: by a Liquibase changelog the platform applies, by the image itself, or not at all.
Grant	A secret a process may read, how it is delivered, and its rotation.
Asset	A configuration file required by a process.
Observability	Where metrics are collected, and the alert level for failures.
Sidcar	A helper process sharing the main process's lifetime.
Capacity	An instance count above one, with the reason it is not derived.

Table 3: Classes of the Project Document

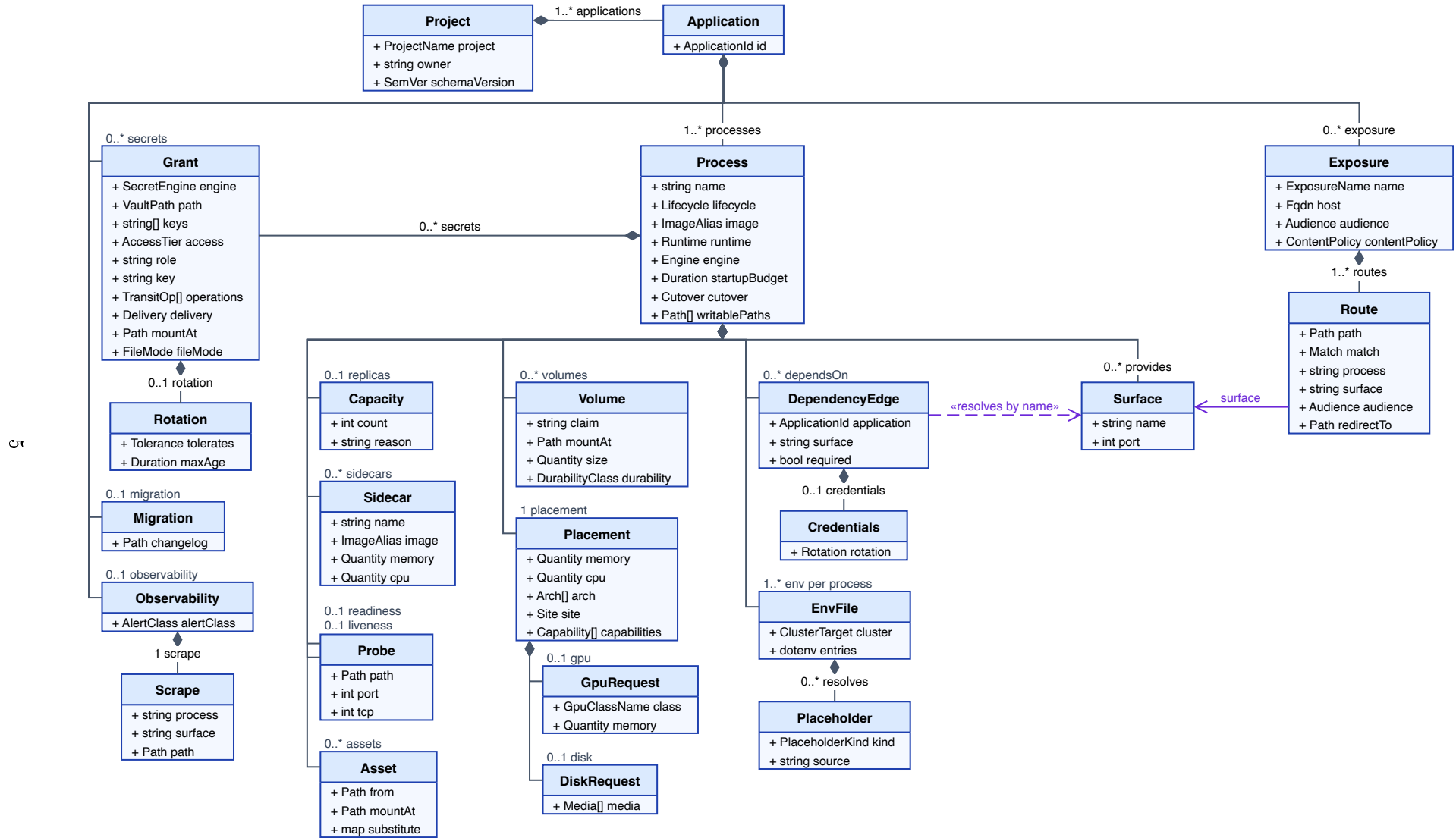


Figure 1: The Project Document

5.2 The Platform Document

The platform document specifies shared cluster facts and policies. For example, it states whether secrets are encrypted at rest (Section 3.5). A tier refers to its proxy application. The delivery policy refers to delivery applications. Constraints reject secret delivery without encryption and storage requests without matching policies (Section 7).

Class	What it represents
Platform	One document per cluster, with its owner and container security restrictions.
PlatformMetadata	Which cluster and project the document describes, and the digest of the machine contract (the facts each machine publishes) it was written against.
Substrate	Cluster facts that other decisions depend on: its version, datastore, server count, whether secrets are encrypted at rest, and its network plugin and policy controller.
Bootstrap	What must exist before the first generated object can be applied. The document records it, and the generators do not create it.
FluxSource	The source reference of Flux, the tool that fetches the generated files and applies them to the cluster.
RenderedArtifacts	The registry repository each project's generated output is published to, one signed artifact per project.
ArtifactSigner	The keyless identity, an issuer and a subject, whose signature Flux accepts on a published artifact.
VaultState	Whether the secret store is unsealed, the second bootstrap fact.
Tier	An entry point of the edge: its audiences, listener, certificate source, authentication endpoint, and the declared Application that acts as its proxy.
DurabilityPolicies	The three durability classes' policies, held together so a missing one is visible.
DurabilityPolicy	One class's backup window and retention.
OffClusterCopy	Where an irreplaceable volume's copy goes, and the credential that writes it.
EnginePolicies	The backup method per engine, held together for the same reason.
EnginePolicy	One engine's backup method: the alias of an image that performs the backup. The document contains no command or script.
MonitorCadence	How often metrics are collected, and the collection timeout.
ProbeCadence	The cluster's probe timing: period, timeout and the number of failures tolerated.
EphemeralPolicy	The size a declared writable path receives.
Provider	A service the cluster runs that this model does not deploy, with its address and named surfaces, which a dependency may refer to.
MigrationPolicy	The runner image alias, deadline and resources for managed migrations.
DeliveryPolicy	The <i>delivery machinery</i> : the applications that switch other applications to a new version and are not switched themselves.
AnalysisPolicy	How often a new version is checked, how many passing checks promote it, and how many failing ones roll it back.
HandoverLedger	Projects on the old path and projects delivered from pinned artifacts. It also records the old path's removal date.

Table 4: Classes of the Platform Document

Figure 2 shows the classes in Table 4 and their relationships.

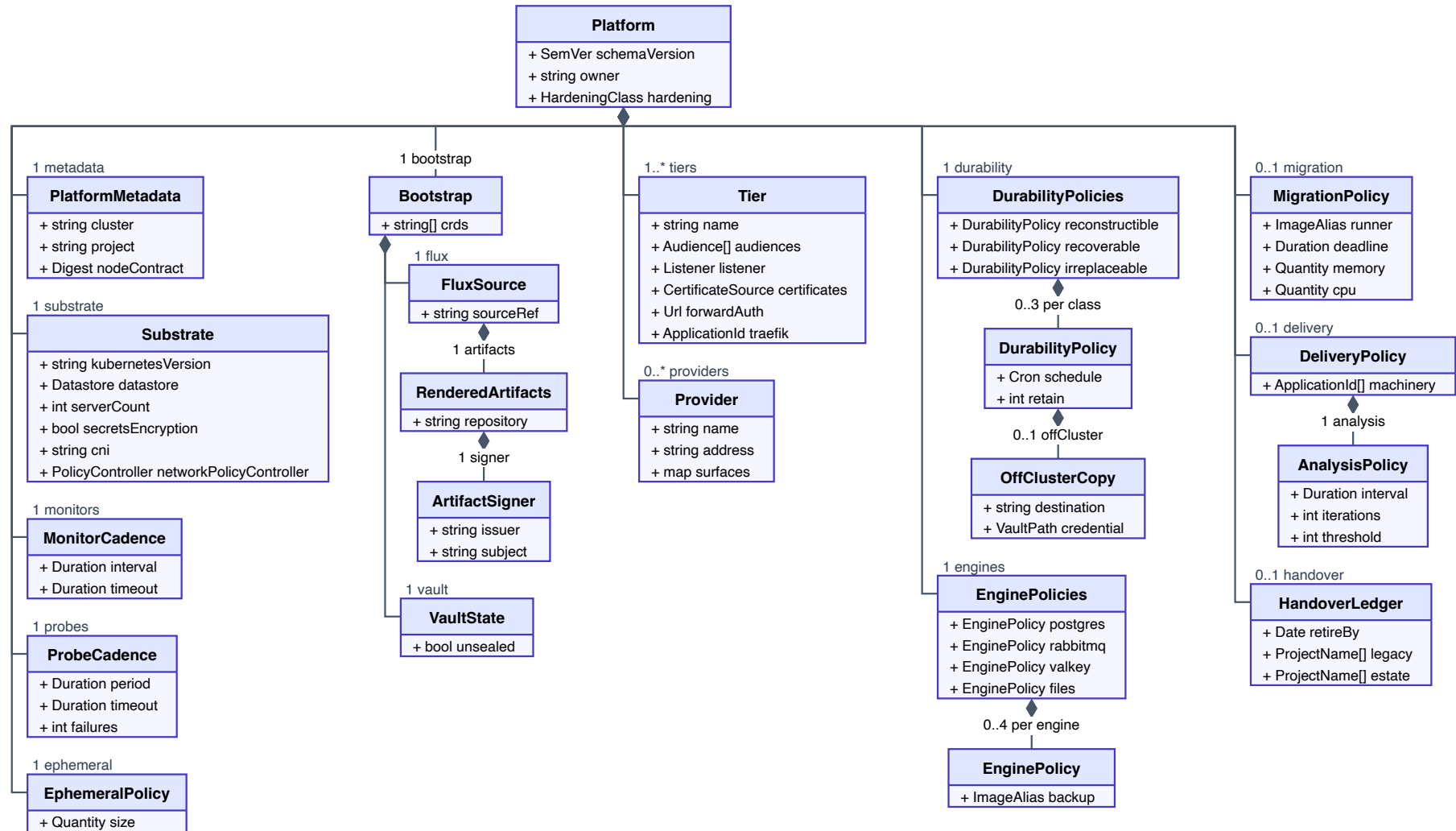


Figure 2: The Platform Document

5.3 Enumerations

Figures 1 and 2 show fixed value sets as types such as `Cutover`, `DurabilityClass` and `Listener` (Section 3.6).

6 The Target Metamodel

The target metamodel is `model/resolved-deployment.ecore` in the archive. Task 2 produces its instances. Task 3 generates files from them. A resolved model records platform decisions, typed resources and output paths.

6.1 Required Content

Generators use the resolved model (Section 3.2).

Decisions. Derived values include edge tiers, eligible machines, process identity, image digests, probe timings, backup plans and network peers. *Switchover* specifies how a new process version replaces the old: **blue-green**, **stop-start** or, for delivery machinery, **rolling**.

Order and atomicity. *Reconcile units* group objects Flux applies together. Dependency edges order these groups. For a blue/green application, the *release gate* controls when the new version receives traffic. Its inputs include members, checks, an interval and a deadline.

Release identity. An application's *revision* hashes its resolved element, excluding the revision field. Migrations and release gates use that revision to identify the same release.

Provenance. A run hash and input digests identify project documents, platform policy, the machine contract, pinned image digests and observed cluster state. Identical inputs must produce identical output.

6.2 Structure

Figure 3 shows the metamodel. The root class, `ResolvedDeployment`, covers all applications (Section 4.3). It contains the provenance, the resolved applications, the reconcile units and the path plan.

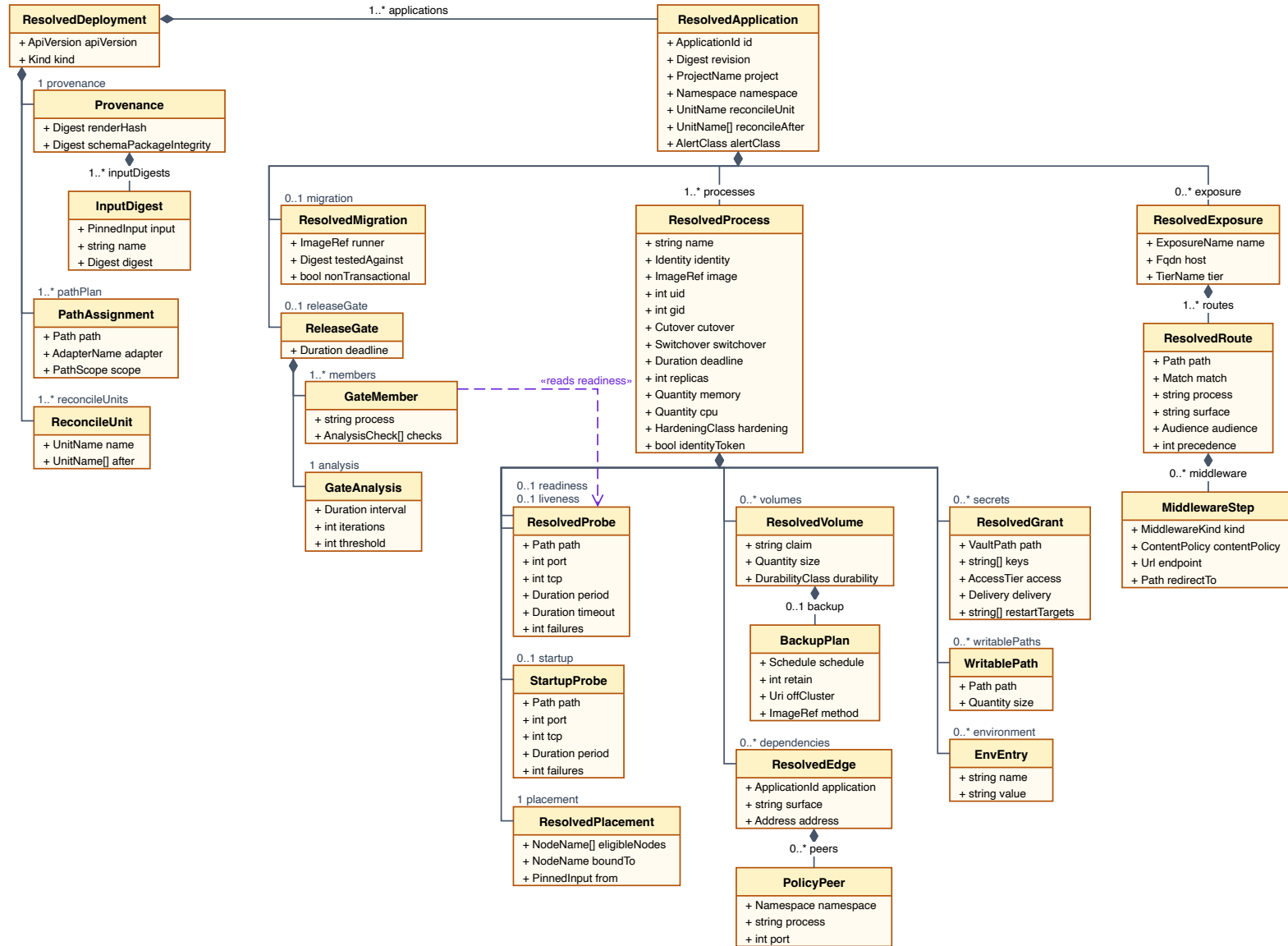


Figure 3: The Target Metamodel

A `ResolvedApplication` also forms the projection published to its repository. Only that published projection has separate provenance.

`ReleaseGate` is absent without blue/green switching. Interrupted applications stop first. Delivery machinery updates in place. `ResolvedMigration` records changesets that cannot run in a transaction and cannot be undone automatically. Migration identity, owner role and deadline follow from the application and platform document.

The `cutover` and `switchover` of a `ResolvedProcess` are unset for a prepare process, which replaces no running version.

6.3 The Path Plan

The *path plan* assigns each generated object to a generator and output path (Section 3.2). Earlier generator-chosen paths duplicated a group object across applications and placed a cluster object in an arbitrary namespace. The plan checks path collisions before generation.

6.4 The Generated Resources

Six generators produce workloads, network policy, monitoring, edge routing, secret-store policies and secret delivery (Table 1). A *Canary* lets Flagger switch traffic to a new version.

Typed resources. The output consists of Kubernetes, Flux, Flagger, Traefik and Vault files. Decision classes specify their content. Typed resource classes specify output paths and generators. They include `WorkloadFile`, `SwitchoverFile`, `NetworkPolicyFile`, `MonitorFile` and `IngressRouteFile`. The remaining types are `SecretFile`, `IdentityFile`, `NamespaceFile` and `IndexFile`. A model-to-text template writes each typed resource.

6.5 Keeping Target Field Names Out of the Model

Under Section 3.5's rule, `SwitchoverFile` records a blue/green switch. Its generator writes a *Canary*. The model also records hardening, writable paths and middleware steps instead of target syntax. A check confines target field names to generators.

7 Implementation in EMF

Three Eclipse bundles under `emf/` implement the metamodels. Maven and Tycho build them on JDK 21. The `metamodel` bundle contains `Ecore`, `Genmodel` and `Complete OCL` files. The `syntax` bundle contains two Xtext grammars and the editor. The `cli` bundle parses and validates authored files. `./mvnw verify` tests the bundles without Eclipse.

7.1 The Concrete Syntax

Two Xtext grammars define the authored syntax (Section 4.2). Both return registered metamodel classes. The pipeline selects the grammar by filename: `platform.intent.yml` or `*.project.yml`.

The syntax uses the YAML subset in the existing configuration.

Indentation as tokens. Preprocessing turns indentation into `BEGIN` and `END` tokens. One rule then handles blocks and flow mappings and sequences.

Unordered keys. Unordered groups accept mappings regardless of key order.

Names as cross-references. The linker resolves references from routes to processes and surfaces. An undeclared metrics process produces a linking error (`E_UNKNOWN_PROCESS`). Handover project identifiers remain strings checked by an invariant because projects are separate documents.

7.2 The Constraints

Complete OCL defines 36 invariants and 9 `def` helpers for the source metamodel. The invariants use 28 distinct codes. Three occur on multiple context classes.

Each invariant has its diagnostic code as its name and the rejected object's class as its context. Diagnostics identify that object with an RFC 6901 JSON Pointer. Table 5 samples the scopes. Appendix B

lists all 28 codes.

Code	Context	What it rejects
E_CUTOVER_UNHONOURABLE	Process	One object: a continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	One document: an application whose processes have different cutover values, although it switches as one unit.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	The platform document: a tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	Across documents: an audience no edge tier in the platform document serves.

Table 5: One Constraint for Each Scope

Ten project-context constraints use another document: the platform document or a database provider’s project file. The invariant’s context identifies the object to fix. `allInstances()` searches the loaded resource set. Guards skip cross-document checks when a project file is loaded alone (Example 1).

```
context Exposure
  inv E_NO_TIER_FOR_AUDIENCE:
    Tier.allInstances()->notEmpty() implies
      Tier.allInstances()->exists(audiences->includes(self.audience))
```

Example 1: E_NO_TIER_FOR_AUDIENCE, with a guard for a project file loaded alone

EMF reads an unset enumeration as its first literal. Optional enumerations use an unwritable **absent** literal, which invariants test instead of **null**.

The pipeline checks constraints requiring all project documents, such as duplicate application identifiers, when it combines the documents.

7.3 How an Invalid Model Is Reported

The pipeline processes three stages, stopping after the first stage with errors. Each internal diagnostic has a code, document name, object path and message (Appendix A.3).

First, parsing reports syntax errors as **schema**, with a line number.

Second, EMF’s **Diagnostician** applies Complete OCL to each document. Invariant names become codes. Containment features and indices form paths. The pipeline reports all violations and unresolved references, including E_UNKNOWN_ codes.

Third, if the individual files pass, the pipeline loads their set together and checks cross-document constraints and references.

A rejected case writes **diagnostics.json** and exit code 1. An accepted case writes **intent.json** and exit code 0. The output files store the exit code in **exit**. Diagnostic JSON contains **code**, **document** and **path**. The internal message is omitted. Both outcomes also write XMI for each parsed document: **notes.project.yml** becomes **notes.project.xmi**. References between XMI files use containment paths. The Xtext editor marks OCL violations on source lines.

The suite compares five accepted examples and 33 rejection cases in **refusals/** with committed oracles: 18 single files and 15 document sets. The production implementation uses the same examples.

8 Example Models

Table 6 lists five accepted source models. Appendix A.4 gives their paths and Section 7 describes the rejected cases.

Appendix A traces three examples through authored, resolved and generated stages.

Example	What it tests
minimal	The smallest complete model: one project, one application, one process, and only required values.
auth	Two continuous processes in one application, so the release gate has two members and its deadline is the maximum over the members. It also has a managed migration, because the application uses a database.
data	Several applications in one project share a namespace. Each volume's durability class determines its backup plan.
knowledge	Two applications with different cutover values: a continuous API that migrates the project database with a changelog, and an interrupted worker with a volume. The worker has no migration. The example also has dependencies, shared secrets and routes with different audiences.
delivery	Two applications of the delivery machinery: Flagger and the release gate. Their processes are continuous but are updated in place instead of switched blue/green, because the release gate cannot control its own release.

Table 6: The Example Models and What Each Tests

8.1 The Minimal Model

The minimal model omits rollout strategy, resource limits, security context, labels, selector, namespace, probe timings and instance count. Platform rules derive these values or reject omissions that cannot be derived.

8.2 A Resolved Model as a Deployment View

Figure 4 shows the **knowledge** resolved model as a deployment view (Section 2.3). It includes two processes, their machines, a volume, the edge tier, a hostname and two platform services.

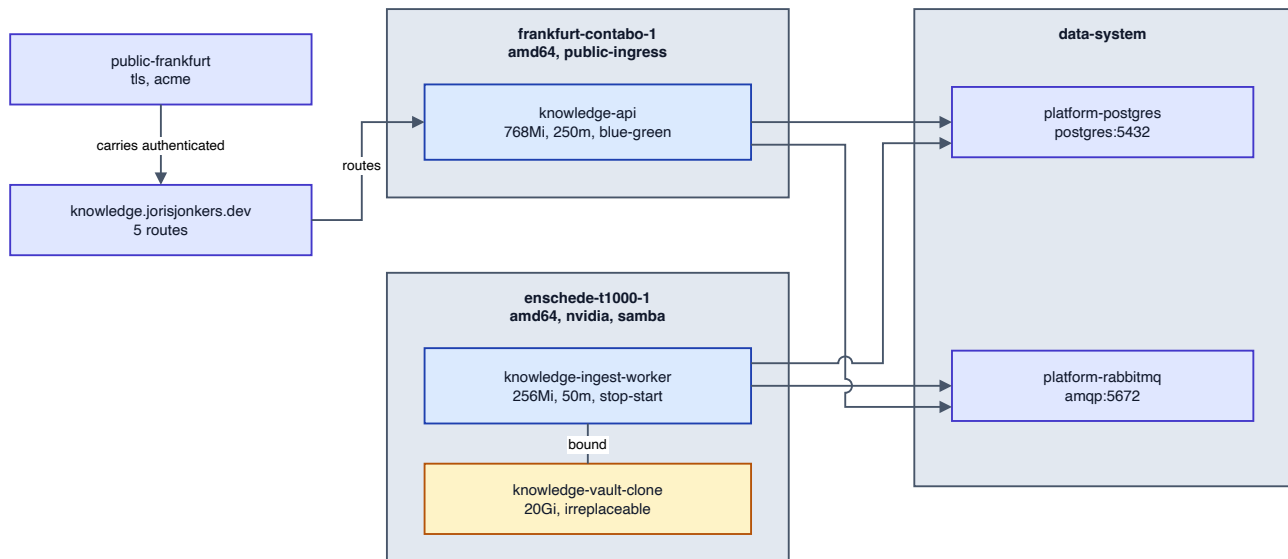


Figure 4: The knowledge Example's Resolved Model as a Deployment View

8.3 Example Target Models

The hand-written `minimal.resolveddeployment` is the expected Task 2 output and an input for testing Task 3 templates.

A Worked Examples

This appendix gives file paths for three examples. Paths under `examples/` and `refusals/` are in the archive (Appendix A.4), and paths under `spec/v1/examples/` are in the repository.

A.1 The Minimal Model, End to End

The input is `examples/minimal/notes.project.yml`. EMF does not write attributes that hold their default value, so `lifecycle` and `alertClass` are missing from `notes.project.xmi` beside it. Table 7 compares what the author wrote with what the hand-written resolved model, `minimal.resolveddeployment`, derives (Section 8). The generated output in `spec/v1/examples/minimal/rendered/` has nine files. In `apps/notes/canary.yaml`, the Canary's values come from the resolved model.

Authored	Derived in the resolved model
<code>project: notes</code> <code>cutover: continuous</code>	<code>namespace="notes-system"</code> , and the reconcile unit (Section 6) <code>switchover="blue-green"</code> , and a <code>releaseGate</code> with this process as its one member
<code>startupBudget: 20s</code> <code>runtime: node</code>	a startup probe of 4 checks 5 seconds apart, and a deadline of 60 seconds the checks <code>error-rate</code> and <code>latency</code> , because the profile exposes HTTP server metrics
<code>image: notes-api</code> <code>audience: anonymous</code>	the image digest, and the numeric user and group it runs as the edge tier <code>public-frankfurt</code> , and a middleware chain without an authentication check
<code>placement</code>	all seven machines are eligible, with no further placement restriction

Table 7: The Minimal Model: Authored and Derived Values

A.2 Continuous and Interrupted Applications in One Project

The input is `examples/knowledge/knowledge.project.yml`, with the two applications of Table 6. In the production implementation's resolved model, under `spec/v1/examples/knowledge/expected/`, the `continuous` application (`resolved.json`) has a release gate and a migration. The `interrupted` one (`resolved.knowledge-ingest.json`) has neither (Section 6). In `rendered/apps/knowledge/migration.yaml`, the migration becomes two Jobs named after the revision: one that migrates the schema, and one that would undo it. Both are generated suspended, so only the release gate can start them.

A.3 A Rejected Model

`refusals/prepare-process-serves.project.yml` declares a prepare process with a port, probes, replicas and a `cutover`, which only a serving process has, and `prepare-process-serves.project.xmi` is its XMI. Its expected diagnostics, `spec/v1/examples/refusals/prepare-process-serves.diagnostics.json`, hold one entry: the code `E_PREPARE_PROCESS_SERVES`, the document, and the path `/applications/0/processes/1`.

A.4 The Hand-in Archive

The archive is an Eclipse project with OCL and Xtext natures. It contains the metamodels, five accepted examples and 33 rejection cases: 18 single files and 15 document sets. The report is separate from the zip. Table 8 gives the layout. Each authored YAML file appears beside its parsed XMI, with any referenced `env/` files.

Each XMI declares its metamodel in `xsi:schemaLocation`, allowing Eclipse to open it without a registered package. The archive's `project-intent.ecore` embeds the OCL invariants and helpers. Plain Validate reports single-document codes. For cross-document sets, open the project XMI, add the platform XMI with Load Resource, then select OCL > Load Document and load `model/project-intent.ocl` before validating. The platform example is parsed alone, leaving six application references unresolved. The archive's genmodels generate model code into its project and other code into separate plug-in projects. Only `minimal` includes a resolved XMI. Task 2 will produce the others. Generated trees and JSON oracles remain in the repository under `spec/v1/examples/`.

Path	What it holds
<code>model/</code>	<code>project-intent.ecore</code> with <code>project-intent.ocl</code> , the constraints of Section 7, and <code>resolved-deployment.ecore</code> , each metamodel with its <code>.genmodel</code>
<code>examples/<name>/</code> <code>examples/minimal/</code>	the authored <code>*.project.yml</code> , its <code>env/</code> files, and its XMI, <code>*.project.xmi</code> also <code>minimal.resolveddeployment</code> , the resolved model as an instance of the target metamodel
<code>examples/platform/</code> <code>refusals/</code>	the platform document, <code>platform.intent.yml</code> , and its XMI 18 single-file cases and 15 document sets, each with authored files and XMI. <code>README.md</code> gives codes and reproduction steps

Table 8: Layout of the Hand-in Archive

B All Constraints

Table 9 lists each invariant of Section 7 by the code it reports, with its context class.

Code	Context	What it rejects
E_ALERT_CLASS_WITHOUT_SIGNAL E_CUTOVER_UNHONOURABLE	Observability Process	An alert level is set, but no metrics are collected. A continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_CUTOVER_MISSING	Process	A process with no cutover value at any level. A prepare process is exempt, because it replaces no running version.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	An application whose processes have different cutover values, although it switches as one unit.
E_PREPARE_PROCESS_SERVES	Process	A prepare process with a port, probes, replicas or a cutover , which only a serving process has.
E_SHARED_DECLARATION_DUPLICATED	seven classes	A shared declaration repeated unchanged at a lower level, where it has no effect.
E_SHARED_QUANTITY	Placement	A memory or CPU quantity declared above process level, where it would mean something different for each process.
E_PLACEMENT_INCOMPLETE	Process	A process with no memory or CPU quantity of its own. No higher level may supply one.
E_OWNER_ROLE_GRANTED	DatabaseGrant	A hand-written grant of the database role that changes the schema. The platform derives this role for the migration only.
E_MIGRATION_OWNER_DUPLICATED	Migration	A second application in one project that declares a migration for the project's database.
E_ENGINE_WITHOUT_DURABILITY	Process	A backup engine declared where no volume's durability class requires a backup.
E_DURABILITY_WITHOUT_ENGINE	Volume	A volume that requires a backup, with no engine to state the backup method.
E_ENV_CANNOT_RELOAD	Rotation	A secret that tolerates a reload but is delivered as an environment variable, which is read only at start.
E_ILLEGAL_DELIVERY_FOR_ACCESS	KvGrant	custody delivered as an environment variable or file, although its paths appear only at runtime.
E_NON_KV_DELIVERY	TransitGrant, DatabaseGrant	Also self-renew delivered as an environment variable, which its identity cannot read. A transit or database grant delivered as an environment variable or file. A transit key never leaves the secret store, and a database credential is issued per lease, so the application fetches both itself.
E_DUPLICATE_ROUTE_MATCH	Route	Two routes of one exposure with the same path and match type, which the derived precedence cannot order.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	A tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_HANDOVER_BOTH_PATHS	HandoverLedger	A project listed on both delivery paths, so two sources would both apply it.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	An audience no declared edge tier serves, so the route reaches no edge.
E_NO_ENGINE_POLICY	Process	An engine with no backup method in the platform document.
E_NO_DURABILITY_POLICY	Volume	A durability class with no policy in the platform document.
E_SECRETS_AT_REST_REQUIRED	Grant	A secret written into a cluster that does not encrypt secrets at rest.
E_MIGRATION_UNDECLARED	Application	An application that uses a database but declares no migration.
E_MIGRATION_WITHOUT_DATABASE	Migration	A migration on an application that uses no database, checked only when the documents of all its providers are loaded.
E_CREDENTIALS_WITHOUT_DATABASE	Credentials	A rotation override on a dependency whose provider has no database.
E_NO_MIGRATION_POLICY	Migration	A changelog, where the platform document defines no migration runner to build it on.
E_NO_DELIVERY_POLICY	Application	A continuous application, where the platform document defines no check interval for its blue/green switch.
E_HANDOVER_UNLISTED	Project	A project that the handover ledger lists on neither delivery path.

Table 9: All Constraints, Their Context Classes and What They Reject